

```

---
title: "HW01"
author: "Jonathan Morris"
date: "2026-09-25"
output: github_document
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

## Load Packages
```{python}
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

```{r}
library(dplyr)
library(tidyverse)
library(ggplot2)
```

# Question 7:

## (a) Read the CSV with ID as text. Preserve the severity order Mild < Moderate < Severe.
Print the dataset dimensions, duplicate-ID count, missing-wait count, and negative-wait
count. (2 points)
```{python}
Read data:
d = pd.read_csv("homework1_clinic.csv", dtype={"id": "string"})

Preserve severity order:
ordered_levels = ["Mild", "Moderate", "Severe"]
d["severity"] = pd.Categorical(d["severity"], categories = ordered_levels, ordered = True)
d_sorted = d.sort_values(by = "severity")
print(d_sorted)

Print data shape:
print(f"Data shape: {d.shape}")

Print duplicated ID's:
duplicated_id = d['id'].duplicated().sum()
print(f"Number of duplicate id's: {duplicated_id}")

Print duplicated waits:
duplicated_wait = d['wait_min'].duplicated().sum()
print(f"Number of duplicate waits: {duplicated_wait}")

Print negative waits:
negative_wait = (d['wait_min'] < 0).sum()
print(f"Number of negative waits: {negative_wait}")
```

## (b) For observed waits, report n, mean, median, sample variance and SD, Q1, Q3, IQR,
fences, and flagged values. Use ddof=1 and NumPy quantile method="linear". Compare the
overlapping results with your hand calculations in Question 4. (3 points)
```{python}
Print n:
print(f"n: {d["wait_min"].size}")

Print the mean:
print(f"Mean: {np.mean(d["wait_min"])}")

```

```

Print the variance:
print(f"Variance: {np.var(d["wait_min"], ddof = 1)}")

Print the standard deviation:
print(f"Standard deviation: {np.std(d["wait_min"], ddof = 1)}")

Print the quartiles:
print(f"Quartiles: {np.nanquantile(d["wait_min"], [0.25, 0.5, 0.75], method = "linear")}")

Find the IQR and fences:
q1 = np.nanquantile(d["wait_min"], [0.25], method = "linear")
q3 = np.nanquantile(d["wait_min"], [0.75], method = "linear")
iqr = q3 - q1
print(f"IQR: {iqr}") # Print IQR
fence1 = q1 - 1.5*iqr
fence2 = q3 + 1.5*iqr
print(f"Fence 1: {fence1}") # print fence 1
print(f"Fence 2: {fence2}") # Print fence 2

Print the flagged values:
flagged = []
for i in d["wait_min"]:
 if i > fence2 or i < fence1:
 flagged.append(i)

print(f"Flagged values: {flagged}")

'''

```

The results are the same as those from question 4!!

## (c) Produce a labeled histogram using edges 0, 5, 10, 15, 20, 25, 30, and print the bin counts. Use bins closed on the left and open on the right, except the final bin includes 30. Also produce a labeled horizontal boxplot with the  $1.5 \times \text{IQR}$  rule. (3 points)

```

Create Histogram:
'''{python}
custom_bins = np.array([0, 5, 10, 15, 20, 25, 31])

plt.hist(d["wait_min"], bins=custom_bins, edgecolor='black', color='blue')

Add tick labels:
centers = custom_bins[:-1] + 2.5 # Label bin centers except for the upper bound.
labels = [f"{b}-{b+4}" for b in custom_bins[:-1]] # Create a list of bin labels.
labels[-1] = "25-30" # Add a last label for the final bin.
plt.xticks(centers, labels) # Add the labels to the plot.

plt.title("Histogram of Wait Times")
plt.xlabel("Wait time (min)")
plt.ylabel("Count")
plt.show()
'''

Create Boxplot:
'''{python}
plt.boxplot(d["wait_min"].dropna(), orientation="horizontal",
whis=1.5, showmeans=False)
plt.title("Boxplot Showing Distribution of Wait Times")
plt.xlabel("Wait (minutes)")
plt.show()
'''

```

## (d) In two or three sentences, describe the distribution, explain why the mean uses the observed-wait count, and state how you would handle the flagged wait. (2 points)

The mean uses the observed wait count in the basic formula for sample mean by dividing the sum of all the non-na observations by the number of non-na values (i.e., the observed wait count). In this case, since the flagged wait is so much higher than other waits, I would run analyses with and without the outlier to see if results changed. If I were presenting or writing a paper, I would report both results. I would also consider using the median because it is often less susceptible to impacts from extreme values.

# Question 8 An observational comparison in R

## a. Read the same CSV. Print the program-by-improved count table. For each program group, report total participants, observed improvement outcomes, number improved, improvement proportion, and number of missing waits. Do not exclude an observed improvement outcome because its waiting time is missing. (3 points)

```
```{r}
# Load the data:
d <- read_csv("homework1_clinic.csv")

# Print program by improved count table:
d |>
  group_by(program, improved) |>
  summarize(count = n())
```

Report results for Program == Yes group:
```{r}
d_yes = d |>
  filter(program == "Yes")

# Report total participants:
total_pp = d_yes |>
  filter(!is.na(id)) |>
  count()

paste0("The Yes program has ", total_pp, " unique participants.")

# Report observed improvement outcomes:
improved_obs = d_yes |>
  filter(!is.na(improved)) |>
  count() |>
  pull()

paste0("The Yes program has ", improved_obs, " observed improvement outcomes.")

# Report number improved:
improved_count = d_yes |>
  filter(improved == 1) |>
  count() |>
  pull()

paste0("The Yes program has ", improved_count, " participants that improved.")

# Report improvement proportion:
improved_proportion = improved_count / improved_obs

paste0("Our improvement proportion for the Yes program is: ", improved_proportion)

# Report number of missing waits:
missing_waits = d_yes |>
  filter(is.na(wait_min)) |>
  count() |>
  pull()
```

```

paste0("The Yes group has ", missing_waits, " missing wait times.")
```
Report results for Program == No group:
```{r}
d_no = d |>
  filter(program == "No")

# Report total participants:
total_pp = d_no |>
  filter(!is.na(id)) |>
  count()

paste0("The No program has ", total_pp, " unique participants.")

# Report observed improvement outcomes:
improved_obs = d_no |>
  filter(!is.na(improved)) |>
  count() |>
  pull()

paste0("The No program has ", improved_obs, " observed improvement outcomes.")

# Report number improved:
improved_count = d_no |>
  filter(improved == 1) |>
  count() |>
  pull()

paste0("The No program has ", improved_count, " participants that improved.")

# Report improvement proportion:
improved_proportion = improved_count / improved_obs

paste0("Our improvement proportion for the No program is: ", improved_proportion)

# Report number of missing waits:
missing_waits = d_no |>
  filter(is.na(wait_min)) |>
  count() |>
  pull()

paste0("The No group has ", missing_waits, " missing wait times.")
```

```

## b. Calculate the Yes-minus-No difference in improvement proportions. Express it as both a proportion difference and percentage points. (2 points)

```

```{r}
# Calculate yes count:
improved_yes_c = d |>
  filter(program == "Yes", improved == 1) |>
  count() |>
  pull()

# Calculate no count:
improved_no_c = d |>
  filter(program == "No", improved == 1) |>
  count() |>
  pull()

# Calculate ratios:
yes_ratio = improved_yes_c / d |> filter(program == "Yes") |> count() |> pull()
no_ratio = improved_no_c / d |> filter(program == "No") |> count() |> pull()

```

```

# Calculate difference:
paste0("The yes-no difference proportion is: ", yes_ratio - no_ratio, ".")
paste0("The yes-no difference percentage is: ", (yes_ratio - no_ratio)*100, "%")
```

c. Make a labeled bar chart of the two improvement proportions with a vertical axis
from 0 to 1. (1 point)
```{r}
# Make a data frame with the proportions:
df = tibble(
  program = c("Yes", "No"),
  ratio = c(yes_ratio, no_ratio)
)

df |>
  ggplot(aes(x = program, y = ratio)) +
  geom_col() +
  ylim(0, 1) +
  xlab("Program") +
  ylab("Improvement Proportion") +
  ggtitle("Improvement Proportions by Program")
```

d. Write a defensible interpretation limited to these participants. Explain why the
difference is not automatically a causal effect, naming one plausible baseline confounder
and how it could affect both program choice and improvement. (2 points)
Based on these results, we can conclude that in this sample of 12 participants, those that
chose counseling had almost double the improvement ratio when compared to those students
that did not chose counseling. This is not necessarily a causal effect because
participants were not randomized and instead self-selected into groups. One possible
confounder is self-selection bias where participants who were more likely to benefit from
counseling could have selected into counseling programs, therefore biasing the outcomes in
number of participants that improved.

Question 9 Checking diagnostic probabilities in Python
In a hypothetical population, disease prevalence is 0.02, sensitivity is 0.90, and
specificity is 0.95. Let D mean disease and T mean a positive test. Assume independent
simulated people and the stated conditional test probabilities.

a. Use np.random.default_rng(813109) and B=100000. First generate B independent
uniforms to assign disease; then generate B new independent uniforms to assign positive
tests with probability 0.90 for people with disease and 0.05 otherwise. Print the four
counts TP, FN, FP, and TN, with clear labels. (3 points)

```{python}
# Initialize rng and B:
rng = np.random.default_rng(813109)
B = 100000

# Create disease vector of T/F
D = rng.random(B) < 0.02

# Create test vector:
p_pos = np.where(D, 0.90, 0.05) # Creates an array which assigns 0.90 to any row with TRUE
in D and 0.05 to FALSE, etc.
T = rng.random(B) < p_pos

# Calculate counts:
TP = np.sum(D & T)
FN = np.sum(D & ~T)
FP = np.sum(~D & T)
TN = np.sum(~D & ~T)

```

```

print(f"True positives (TP): {TP}")
print(f"False negatives (FN): {FN}")
print(f"False positives (FP): {FP}")
print(f"True negatives (TN): {TN}")
```

```

## b. Calculate empirical sensitivity, specificity, PPV, and NPV from their appropriate subsets. Compute the corresponding theoretical probabilities using the given model and compare theory with simulation. Briefly explain any differences; simulation is an empirical check rather than a mathematical proof. (3 points)

```

```{python}
# Calculate sensitivity, the proportion of people that test positive given that they have
the disease:
sensitivity = TP/(TP + FN)
print("The empirical sensitivity is: ",sensitivity)
print("The theoretical sensitivity is: ",0.90, "as specified in the problem setup")

# Calculate specificity, the proportion of people that test negative given that they do
not have the disease:
specificity = TN/(TN + FP)
print("The empirical specificity is: ", specificity)
print("The theoretical specificity is: ",0.95, "as specified in the problem setup")
```

```

Observe the following equations adapted from Bayes rule used to calculate theoretical PPV and NPV:

Let  $T^+ =$  positive test,  $T^- =$  negative test,  $D^+ =$  disease present, and  $D^- =$  disease absent.

$$\text{PPV} = P(D^+ \mid T^+) = \frac{P(T^+ \mid D^+) \cdot P(D^+)}{P(T^+ \mid D^+) \cdot P(D^+) + P(T^+ \mid D^-) \cdot P(D^-)}$$

$$\text{NPV} = P(D^- \mid T^-) = \frac{P(T^- \mid D^-) \cdot P(D^-)}{P(T^- \mid D^-) \cdot P(D^-) + P(T^- \mid D^+) \cdot P(D^+)}$$

```

```{python}
# Calculate positive predictive value, the proportion of people that actually have the
disease given that they test positive:
PPV = TP/(TP + FP)
print("The empirical PPV is: ", PPV)
print("By bayes rule, the theoretical PPV is: ", "[P(T+ | D+) * P(D+)] / [P(T+ | D+) *
P(D+) + P(T+ | D-) * P(D-)] = ", (0.9*0.02)/((0.9*0.02) + ((1-0.95)*0.98)))

```

```

# Calculate negative predictive value, the proportion of people that test negative and do
not have the disease:
NPV = TN/(TN + FN)
print("The empirical NPV is: ", NPV)
print("By bayes rule, the theoretical NPV is: ", "[P(T- | D-) * P(D-)] / [P(T- | D-) *
P(D-) + P(T- | D) * P(D)] = ", (0.95*0.98)/((0.95*0.98) + ((1-0.90)*0.02)))

```

The results closely resemble what we would expect from a theoretical calculation, with only slight differences in the computed sensitivity and specificity values. These differences are largely accounted for when we round to the second decimal place, but demonstrate that the simulation will still have slight discrepancies when compared to the

theoretical case. As B goes to infinity, our theoretical and simulated results will get closer and closer. This does not demonstrate a mathematical proof because we only simulated the results in a heuristic example. To prove these relationships, we would need to show that as B goes to infinity, the simulated result converges to the theoretical result.

c. Holding sensitivity and specificity fixed, calculate the theoretical PPV when prevalence rises to 0.20. Explain the direction of the change. (2 points)

```
```{r}
paste0("By bayes rule, the theoretical PPV with prevalence at 0.20 is: ",
(0.9*0.2)/((0.9*0.2) + ((1-0.95)*0.8)))
```
```

The PPV rises to 0.82. See equations in part (b) for details on how this is calculated. The rise is likely due to the fact that as the prevalence rises, we expect more people in the population to have the disease, hence it is also much more likely that someone who tests positive will have the disease also.

Question 10 Sampling without replacement in R

A group contains eight distinct people labeled 1–8; people 1–3 have a specified allergy. In one trial, select three people uniformly without replacement. The event of interest is that all three selected people have the allergy. Reset the full eight-person population before the next trial.

a. State the exact event probability as a product of conditional probabilities. Also calculate the corresponding probability for three independent draws with replacement. (2 points)

Let x_1, x_2, x_3 be binary random variables representing the outcome of the first, second, and third draws respectively where:

$$x_i = \begin{cases} 0 & \text{if person } i \text{ has no allergies} \\ 1 & \text{if person } i \text{ has allergies} \end{cases}$$

Now let Y define the sum of random variables x_1, x_2, x_3 , so Y represents the number of successes in one trial and Y follows a hypergeometric distribution.

Based on this definition, our event of interest is the event when $Y = 3$:

Thus, $P(Y = 3) = P(x_1 = 1) + P(x_2 = 1 \mid x_1 = 1) + P(x_3 = 1 \mid x_2, x_1 = 1) = \frac{3}{8} * \frac{2}{7} * \frac{1}{6} = 0.01785714\$.$

In the independent case with replacement, our count resets at each draw, so the probability is the same across all draws:

$$P(Y = 3) = P(x_1 = 1) + P(x_2 = 1) + P(x_3 = 1) = \frac{3}{8} * \frac{3}{8} * \frac{3}{8} = 0.05273438\$.$$

b. Use B=100000 and set.seed(813110) to simulate groups with sample.int(8, 3, replace=FALSE). Then use set.seed(813111) and replace=TRUE for the second model. Report successes, empirical probabilities, and theoretical probabilities for both models. (3 points)

```
```{r}
B = 10000
set.seed(813110)
Run the first model without replacement:
m1 <- replicate(B, all(sample.int(8, 3, replace = FALSE) <= 3))
succ1 <- sum(m1)
emp1 <- mean(m1)
theo1 <- (3/8) * (2/7) * (1/6)
```

```
paste0("Successes for model 1: ", succ1)
```

```

paste0("Empirical probability for model 1: ", emp1)
paste0("Theoretical probability for model 1: ", theo1)

set.seed(813111)
Run the second model with replacement:
m2 <- replicate(B, all(sample.int(8, 3, replace = TRUE) <= 3))
succ2 <- sum(m2)
emp2 <- mean(m2)
theo2 <- (3/8)^3
paste0("Successes for model 2: ", succ2)
paste0("Empirical probability for model 2: ", emp2)
paste0("Theoretical probability for model 2: ", theo2)
```



## c. For the simulation without replacement, report running empirical probabilities after 100, 1,000, 10,000, and 100,000 trials. Plot the running probability against the number of trials, with a horizontal line at the theoretical probability. A logarithmic horizontal axis is acceptable. (3 points)



```

```{r}
# Set Seed:
set.seed(813110)

# Initialize data frames:
trials = c(100, 1000, 10000, 100000)
probs_df = data.frame(B = c(100, 1000, 10000, 100000),
                      probs = c(NA, NA, NA, NA),
                      log_B = log(B))

# Run the simulations:
i = 1
for (B in trials) {
  # Run Model
  m1 <- replicate(B, all(sample.int(8, 3, replace = FALSE) <= 3))
  succ1 <- sum(m1)
  emp1 <- mean(m1)

  # Report results:
  paste0("Successes for model 1 with B = ", B, ": ", succ1)
  paste0("Empirical probability for model 1 with B = ", B, ": ", emp1)
  probs_df[i, 2] = emp1
  i = i + 1
}

# Plot the graph:
probs_df |>
  ggplot(aes(x = log(B), y = probs)) +
  geom_point() +
  geom_line() +
  geom_hline(yintercept = theo1, linetype = "dotted") +
  xlab("Logarithm of Trial Numbers") +
  ylab("Empirical Probabilities") +
  annotate("text", x = 6, y = theo1, label = "Theoretical Probability",
          hjust = 1, vjust = -0.5, color = "red")
```


d. Explain why the two models have different probabilities. Does the law of large numbers guarantee exact agreement after 25 trials, or that each additional trial moves the estimate closer to the truth? (2 points)

The theoretical probability is defined for the case where our sample size B trends to infinity. In theory, if we were to repeat this experiment an infinite number of times, our empirical probability would converge to the theoretical probability. This means that we will not necessarily have exact agreement after 25 trials, and that as we keep sampling greater and greater numbers, we get closer and closer to the theoretical result.


```


```

